

知光项目功能实现与面试改造指南

知识社区后端 · 源码验证版 · 面试改造路线

生成日期	2026-09-09
适用场景	Java 后端 / 高并发业务 / AI 应用方向面试
分析口径	以当前本地源码为准，README 只用于辅助理解；“已有能力”和“建议改造”分开描述。

1. 先说结论

知光是一个面向知识获取与分享的社区后端，业务闭环已经覆盖“账号注册登录 → 内容创作与 OSS 上传 → 发布与 Feed 展示 → 点赞收藏 → 关注关系 → 搜索 → 单篇内容 AI 问答”。项目不是单纯 CRUD，真正有面试价值的部分主要有五组：

- Redis 分片 Bitmap + Lua + Kafka 聚合 + 固定结构 SDS 的点赞/收藏计数系统；
- MySQL 主表 + Outbox + Canal + Kafka 构建关注关系异步投影；
- Caffeine、Redis 片段缓存、热点探测和 SingleFlight 组成的 Feed/详情缓存；
- Elasticsearch 全文检索、业务权重排序、联想词和 search_after 深分页；
- OSS 内容切片、Elasticsearch 向量检索、DeepSeek SSE 输出组成的单篇 RAG 问答。

它目前更像“功能完整、设计意识较强的单体业务系统”，还不能直接包装成经过生产流量验证的分布式系统。面试时应说“为高并发场景做了机制设计和代码实现”，不要说“支撑过百万 QPS”或“线上零故障”。

最值得做的三项魔改是：

- 把发布流程改成可校验、可幂等的内容状态机，并补齐可靠 Outbox；
- 把点赞事件链路改成 Redis 内原子写事实与事件，增加幂等消费、死信和对账；
- 把 RAG 改成带服务端元数据过滤、引用溯源和评测集的可验证问答系统。

若时间更充足，再做“关注流个性化 Feed 的推拉结合”，它会成为最有区分度的后端亮点。

2. 源码核验摘要

项目	核验结果
源码位置	当前工作区 D:\Code\知光正版\zhiguang_be-main
代码规模	126 个主代码 Java 文件，约 9,121 行；11 个 Controller，36 个接口方法，5 个 MyBatis XML Mapper
数据结构	MySQL 表包括 users、login_logs、know_posts、outbox、following、follower
自动化测试	3 个测试类、11 个测试用例；mvn -q test 在 2026-09-09 本地执行通过
测试覆盖边界	主要覆盖 JWT、热点探测和知文缓存失效；计数、Outbox、Kafka、关系、搜索、RAG 暂无集成测试
运行配置	src/main/resources/application.yml 当前为空；仓库可以编译和运行单元测试，但完整启动所需的 MySQL、Redis、Kafka、Canal、ES、OSS、模型等配置需要从外部补齐

项目	核验结果
版本信息	当前目录不包含 .git 元数据，因此无法核验分支和 commit；本文基于当前源码快照

3. 项目定位与整体架构

3.1 一句话介绍

知光是一个 Java 21 + Spring Boot 的知识社区后端，通过 MySQL 保存账号与内容事实，Redis 承接缓存、关系列表和互动计数，Kafka/Canal 驱动异步投影，Elasticsearch 同时服务全文搜索和向量检索，并接入 DeepSeek 实现摘要生成与单篇文章流式问答。

3.2 核心数据流

```
■■■
■■ ■■/■■ ■■> Spring Security + JWT ■■> MySQL(users/login_logs) + Redis(refresh token/code)
■■ ■■■■■ ■■> ■■ ■■> OSS ■■■■■ ■■> ■■■■■ ■■> MySQL(know_posts)
■ ■■> Outbox -> Canal -> Kafka -> ES ■■■■■
■ ■■> Markdown ■■ -> ES ■■■■■
■■ ■■ Feed ■■> Caffeine ■■> Redis(ids/item) ■■> MySQL
■■ ■■■■■ ■■> Redis Bitmap + Lua ■■> Kafka ■■> Redis Hash ■■ ■■> SDS ■■
■■ ■■■■■ ■■> MySQL(following + outbox) ■■> Canal/Kafka ■■> follower + Redis ZSet + ■■■■■
■■ AI ■■ ■■> ■■■■■ ■■> Prompt ■■> DeepSeek ■■> SSE ■■■■■
```

3.3 各存储的职责

组件	在项目中的职责	面试时的定位
MySQL	用户、登录审计、知文元数据、关注主表、粉丝投影表、Outbox	核心业务事实与事务边界
OSS	Markdown 正文、图片、头像	大文件不穿过应用服务器，减少带宽与内存占用
Redis	验证码、刷新令牌、Feed 缓存、关系 ZSet、Bitmap、SDS、聚合桶、锁与限流	高频读写和派生状态
Kafka	点赞收藏增量事件、Canal Outbox 事件	异步解耦、削峰和事件回放基础
Canal	订阅 outbox 表 binlog 并桥接到 Kafka	避免业务事务内直接双写 MQ
Elasticsearch	内容全文索引、联想建议、RAG 向量切片	搜索读模型与语义检索
Caffeine	Feed 页、详情页和大 V 列表头部的进程内缓存	吸收热点流量，降低 Redis RTT

4. 功能与对应实现

4.1 账号认证与安全会话

用户能力

- 手机号或邮箱验证码发送;
- 注册、密码/验证码登录、刷新令牌、登出、重置密码;
- 获取当前用户信息和登录审计记录。

实现链路

AuthController → AuthService → UserService / VerificationService / JwtService / RefreshTokenStore

- Spring Security 采用无状态 Resource Server, 除认证、公开 Feed、公开详情和问答接口外, 其余接口默认鉴权;
- Access Token 与 Refresh Token 均为 RS256 JWT, 含 uid、token_type、jti;
- Refresh Token 的 jti 写入 Redis 白名单, 刷新时撤销旧令牌并存入新令牌, 登出可撤销单个令牌, 重置密码可撤销全部刷新令牌;
- 验证码存入 Redis Hash, 带 TTL、发送间隔、日限额和最大尝试次数;
- 密码用 BCrypt, 登录成功/失败写入 login_logs。

源码入口

- src/main/java/com/tongji/auth/service/AuthService.java:87
- src/main/java/com/tongji/auth/token/JwtService.java:52
- src/main/java/com/tongji/auth/token/RedisRefreshTokenStore.java:34
- src/main/java/com/tongji/auth/verification/VerificationService.java:55
- src/main/java/com/tongji/auth/config/SecurityConfig.java:45

可信边界

当前验证码发送器是日志实现, 并非真实短信/邮件通道; 刷新令牌“校验 → 撤销 → 新增”也不是一个原子操作。面试可以讲双令牌和白名单机制, 但不要说已经完成多设备会话管理或刷新令牌重放检测。

4.2 用户资料与头像

用户能力

- 更新昵称、简介、性别、生日、学校、标签和知光号;
- 检查知光号唯一性;
- 上传头像并保存 OSS 地址。

实现特点

- PATCH 请求只更新非空字段，避免未提交字段被覆盖；
- 头像当前由后端接收 MultipartFile 再上传 OSS；
- 更新后重新查询用户快照返回。

源码入口

- src/main/java/com/tongji/profile/service/impl/ProfileServiceImpl.java:67
- src/main/java/com/tongji/storage/OssStorageService.java:49
- src/main/resources/mapper/UserMapper.xml

4.3 知文创作、OSS 直传与发布

用户能力

- 创建草稿；
- 获取 10 分钟有效的 OSS PUT 预签名 URL；
- 前端直传正文/图片后确认 objectKey、ETag、大小、SHA-256；
- 更新标题、标签、摘要、可见性、置顶等元数据；
- 发布、查看详情和软删除。

实现链路

- createDraft 使用雪花算法生成内容 ID，写入 draft 状态；
- StorageController 从 JWT 取用户 ID 并校验草稿归属，按正文/图片场景生成隔离的对象 Key；
- 前端绕过业务服务直传 OSS，后端只保存地址与内容指纹；
- 元数据更新、发布和删除时写 outbox，供搜索索引异步更新；
- 确认正文和发布后尝试预构建 RAG 索引；
- 详情读取采用 Caffeine → Redis → MySQL，并在返回前叠加实时点赞/收藏状态。

源码入口

- src/main/java/com/tongji/knowpost/service/impl/KnowPostServiceImpl.java:125
- src/main/java/com/tongji/storage/api/StorageController.java:62
- src/main/java/com/tongji/storage/OssStorageService.java:90
- src/main/resources/mapper/KnowPostMapper.xml
- src/main/java/com/tongji/knowpost/id/SnowflakeIdGenerator.java:54

可信边界

数据库预留了 draft/published/deleted 等状态，但当前 SQL 没有限制合法前置状态，重复调用发布接口仍可能再次增加作者发文数；followers/school/unlisted 虽能写入字段，详情读取目前只真正区分“公开已发布”和“作者本人”。这正适合升级成状态机与统一权限策略。

4.4 公共 Feed、我的发布与详情缓存

用户能力

- 匿名查看公开已发布内容；
- 登录后查看自己的发布内容；
- 在 Feed 与详情中看到计数以及自己的点赞/收藏状态。

实现机制

- 公共 Feed：Caffeine 页缓存 → Redis ID 列表与 feed:item:{id} 片段 → MySQL 回源；
- 详情：Caffeine 整体对象 → Redis JSON → MySQL 回源；
- 空详情以 NULL 短 TTL 缓存，缓解缓存穿透；
- 同一 Key 未命中时用进程内 ConcurrentHashMap + synchronized 实现 SingleFlight；
- TTL 添加随机抖动，热点探测器根据滑动窗口访问次数延长热点条目 TTL；
- Feed 建立“内容 ID → 页面 Key”的按小时反向索引，内容或计数变化时定位相关本地页缓存。

源码入口

- src/main/java/com/tongji/knowpost/service/impl/KnowPostFeedServiceImpl.java:112
- src/main/java/com/tongji/knowpost/service/impl/KnowPostServiceImpl.java:401
- src/main/java/com/tongji/cache/hotkey/HotKeyDetector.java:55
- src/main/java/com/tongji/knowpost/listener/FeedCacheInvalidationListener.java:66

可信边界

这套方案对单实例或少量实例有意义，但本地热点统计、SingleFlight 和 Caffeine 天然实例级。当前 Redis 片段命中后会把已经叠加用户状态的响应放入公共 Caffeine 页缓存；后续本地命中虽然会再次调用 enrich 重算用户状态，暂未形成直接的跨用户返回错误，但缓存边界不够干净，也造成了额外计算和维护复杂度。改造时应强制只缓存“公共基础对象”，个性化字段永远只在返回前组装，并使用批量接口消除逐条查询计数与 Bitmap 状态的 N+1 调用。

4.5 点赞/收藏与高并发计数

这是当前最值得深入讲的模块。

写链路

- 用户 ID 按 32,768 位一个分片映射到 Bitmap Key;
- Lua 原子执行 GETBIT + SETBIT, 只有 0→1 或 1→0 时才返回 “状态发生变化” ;
- 真正变化才发送 CounterEvent 到 Kafka, 天然避免重复点击造成重复计数;
- 消费者先用 Redis Hash 按实体与指标聚合 delta, 成功后手动提交 Kafka 位点;
- 定时任务每秒将 Hash 增量通过 Lua 原子折叠进固定长度 Redis String/SDS。

读与修复链路

- 正常读取一个固定 20 字节 SDS, 从字段偏移解码点赞和收藏数;
- 批量 Feed 读取使用 Redis Pipeline 降低 RTT;
- SDS 缺失或长度异常时, 在 Redisson 分布式锁、限流和指数退避保护下, 对各 Bitmap 分片执行 BITCOUNT 重建;
- 另有默认关闭的 Kafka 历史回放消费者, 可用于灾难恢复。

源码入口

- src/main/java/com/tongji/counter/service/impl/CounterServiceImpl.java:128
- src/main/java/com/tongji/counter/event/CounterAggregationConsumer.java:57
- src/main/java/com/tongji/counter/event/CounterAggregationConsumer.java:76
- src/main/java/com/tongji/counter/event/CounterRebuildConsumer.java:47
- src/main/java/com/tongji/counter/schema/BitmapShard.java:15

为什么这样设计

- Bitmap 适合回答 “某用户是否已点赞”, 空间和判重成本都比每个用户一个 Redis Key 更低;
- 分片避免超大偏移造成单 Key 过度膨胀;
- 写聚合把大量单次互动折叠为较少的 SDS 更新;
- 固定结构一次 GET 就能取得多个计数, 内存布局紧凑。

可信边界

当前事实状态先写 Redis Bitmap, 再异步调用 Kafka; 如果 Kafka 发送失败, 状态已经变化但增量事件可能丢失, 只能依靠后续重建纠偏。重建还使用 KEYS 枚举 Bitmap/聚合桶, 在大键空间生产环境会阻塞 Redis。这里是可靠事件与脏集合改造的最佳切入点。

4.6 关注关系与用户维度计数

用户能力

- 关注、取关、查询单向/被关注/互关状态；
- 查询关注与粉丝列表，支持 offset 和时间游标；
- 查询关注数、粉丝数、发文数、获赞数、获收藏数。

实现机制

- following 是关注主表；关注或取关时在同一个 MySQL 事务中写 outbox；
- Canal 订阅 outbox binlog 并写 Kafka；关系消费者异步维护 follower 投影表；
- 关注/粉丝列表缓存在 Redis ZSet，以时间戳为 score；大 V 用户额外缓存 Top 500；
- 关系事件更新用户维度 SDS，接口每 300 秒抽样比对 MySQL 关注/粉丝数，发现不一致时全量重建；
- 关注写接口使用 Redis Lua 令牌桶限流。

源码入口

- src/main/java/com/tongji/relation/service/impl/RelationServiceImpl.java:96
- src/main/java/com/tongji/relation/outbox/CanalKafkaBridge.java:92
- src/main/java/com/tongji/relation/processor/RelationEventProcessor.java:37
- src/main/java/com/tongji/counter/service/impl/UserCounterServiceImpl.java:96
- src/main/resources/mapper/RelationMapper.xml

可信边界

设计目标是 Outbox 的最终一致性，但业务代码当前捕获并吞掉 Outbox 插入异常，主表仍可能提交；Canal 桥接发送 Kafka 后没有等待发送结果就确认 Canal 批次。面试时可以说明方案和现有缺口，再给出可靠性改造，而不要直接宣称“严格保证消息不丢”。

4.7 Elasticsearch 搜索

用户能力

- 标题与正文关键词检索；
- 标签过滤、高亮摘要；
- 基于相关性、发布时间、点赞数、浏览数的综合排序；
- search_after 游标深分页；
- Completion Suggester 标题前缀联想。

实现机制

- 索引初始化时为中文标题/正文配置 IK 分词；
- multi_match 对标题设置更高权重，function_score 使用互动字段做加权；
- Outbox 事件驱动内容 upsert 或软删除；
- 应用启动且索引为空时，从 MySQL 分页回灌公开内容；
- 正文从 OSS 拉取，失败时退化为描述。

源码入口

- src/main/java/com/tongji/search/service/impl/SearchServiceImpl.java:61
- src/main/java/com/tongji/search/index/SearchIndexInitializer.java:31
- src/main/java/com/tongji/search/index/SearchIndexService.java:91
- src/main/java/com/tongji/search/outbox/CanalOutboxConsumerSearch.java:34

可信边界

搜索文档中的互动计数只在内容索引更新时取一次快照，点赞事件不会直接更新 ES，所以业务排序权重可能滞后；索引写入方法吞掉异常后，上层消费者仍会提交消息。这两点适合改成“事件驱动特征更新 + 可重试索引任务”。

4.8 AI 摘要与单篇 RAG 问答

用户能力

- 将正文交给 DeepSeek 生成不超过 50 字的中文摘要；
- 围绕单篇公开知文提问，通过 SSE 接收流式答案；
- 手动重建单篇向量索引。

RAG 实现链路

- 从 MySQL 查询文章，限制为 published + public；
- 用 SHA-256 优先、ETag 兜底判断索引版本是否变化；
- 从 OSS 拉取 Markdown，先按标题分段，再按 800 字符、100 字符重叠切片；
- 删除旧切片后，将文本与 postId/chunkId/position/fingerprint 元数据写入 VectorStore；
- 问答前确保索引存在，执行向量宽召回并筛选当前 postId；
- 将上下文与问题组合成受约束 Prompt，通过 DeepSeek 流式输出。

源码入口

- src/main/java/com/tongji/llm/service/impl/KnowPostDescriptionServiceImpl.java:36
- src/main/java/com/tongji/llm/rag/RagIndexService.java:59
- src/main/java/com/tongji/llm/rag/RagQueryService.java:41
- src/main/java/com/tongji/knownpost/api/KnowPostRagController.java:39

可信边界

当前向量检索先在全局索引取 Top N，再由 Java 过滤 postId。数据量变大后，Top N 可能全属于其他文章，导致当前文章没有上下文。系统也还没有引用展示、问答记录、效果评测和成本/限流治理。因此可以讲完整 RAG 链路，但不要宣称已有准确率指标或防幻觉闭环。

5. 面试时最该讲的四条主线

主线一：为什么计数不是直接 `UPDATE count = count + 1`

数据库热行更新会产生锁竞争，而且还要单独保存用户是否点过赞。当前方案把“用户是否点赞”放进分片 Bitmap，以 Lua 原子切换保证接口幂等；计数变化通过 Kafka 异步聚合后写入紧凑 SDS，读侧一次 GET 获取多个字段。异常时可以从 Bitmap 事实重建计数。

追问时主动补充：当前 Bitmap 与 Kafka 之间仍有事件丢失窗口，生产化会用 Redis Lua 同时执行 `SETBIT + XADD`，让状态事实和待投递事件在同一 Redis 原子操作中落地。

主线二：为什么关注关系同时有 following 和 follower

following 服务“我关注了谁”，follower 服务“谁关注了我”。如果只保留一张表，两种方向都能查，但大规模场景下读热点、索引和扩展策略受限。项目将 following 作为主事实，follower 作为异步投影，再用 ZSet 承接列表查询，体现了 CQRS/读模型思想。

追问时不要回避一致性：两张表是最终一致的，Outbox、幂等消费和对账负责收敛；当前代码的异常处理还需加强。

主线三：Feed 缓存为什么要拆成页面 ID 和条目片段

整页缓存会复制大量相同内容，一个条目更新时需要删除很多页面。拆为“页面 ID 顺序 + 可复用条目片段”，能减少冗余，并通过反向索引定位受影响页面。用户自己的 liked/faved 不进入共享缓存，而应在返回前叠加。

追问时说明边界：本地 SingleFlight 只能挡住单实例惊群，多实例需要 Redis 锁或逻辑过期方案；个性化字段必须与公共缓存严格分离。

主线四：全文搜索与 RAG 为什么用两套索引

全文搜索解决关键词召回、过滤、排序、联想和深分页；向量索引解决语义相似检索。两者的数据结构和评价指标不同，不应强行合成一个接口。后续可以在 RAG 中做 BM25 + 向量混合召回，再用 reranker 排序，但业务搜索仍保留独立的结构化排序逻辑。

6. 高频面试问题与参考回答

6.1 项目与架构

Q：这个项目最有价值的地方是什么？ A：不是组件数量，而是围绕知识社区建立了三条可追问的数据链路：互动状态到聚合计数、关注主表到粉丝投影、内容事实到搜索和向量索引。每条链路都有明确的事实源、异步派生数据和修复策略。

Q：这是微服务吗？ A：当前是模块化单体。模块按 auth、knowpost、counter、relation、search、llm 等包拆分，但部署和数据库仍是一个应用。先在单体内把事务边界和模块接口做清楚，达到独立扩缩容需求后再拆，比为了简历强行微服务更合理。

Q：系统的事实源分别是什么？ A：账号、内容、关注事实主要在 MySQL；点赞收藏的用户状态事实目前在 Redis Bitmap；Feed、粉丝表、用户计数、全文索引和向量切片都是可重建或最终一致的派生数据。

6.2 计数系统

Q：Bitmap 为什么要分片？ A：Redis Bitmap 的物理长度由最大 bit 偏移决定，用户 ID 很大或稀疏时单 Key 会膨胀。按固定区间把用户 ID 映射到 chunk 和 chunk 内 bit，可控制单 Key 大小，也便于分批统计和迁移。

Q：Lua 解决了什么问题？ A：它把读取旧状态、判断是否变化、设置新状态放在一次 Redis 原子执行中。重复点赞返回未变化，不产生新的 delta，从源头保证操作幂等。

Q：为什么还要 SDS 汇总，直接 BITCOUNT 不行吗？ A：BITCOUNT 适合校验和重建，不适合每次 Feed 查询都遍历多个分片。SDS 是面向读的紧凑快照，一次 GET 就能取多个计数；Bitmap 是事实，SDS 是派生汇总。

Q：计数能保证不丢吗？ A：当前不能做绝对承诺。Lua 更新 Bitmap 后 Kafka 异步发送存在间隙，Kafka 失败时 delta 可能丢失；系统可通过 Bitmap 重建纠偏。生产化改造会在 Lua 中同步 XADD 到 Redis Stream，再可靠转发或直接由消费组处理，并加对账指标。

6.3 Outbox 与关系系统

Q：为什么使用 Outbox + Canal？ A：业务表和 Outbox 在一个 MySQL 本地事务中提交，避免“数据库成功、MQ 失败”的经典双写问题；Canal 从 binlog 捕获已提交事件，再投递 Kafka。代价是链路更长，

需要监控 binlog 延迟、Kafka 投递和消费幂等。

Q: 消费者如何幂等? A: 当前关系消费者用 Redis SETNX 去重, 粉丝表本身也有唯一键并使用 upsert。更严格的实现会给每个事件稳定 event_id, 在消费者侧用 MySQL inbox 唯一索引记录处理结果, 业务投影和 inbox 同事务提交。

Q: 关注列表为什么用 ZSet? A: ZSet 同时保存成员和时间分数, 天然支持倒序列表和时间游标。相比 offset 深分页, 游标不会因为前面新增数据导致严重漂移, 复杂度也更稳定。

6.4 缓存与 Feed

Q: 怎么处理缓存穿透、击穿和雪崩? A: 不存在的详情用短 TTL 空值缓存处理穿透; 同 Key 回源用 SingleFlight 处理击穿; TTL 随机抖动减少雪崩; 热点探测会延长高频条目的 TTL。当前 SingleFlight 是单机级, 多实例场景还需分布式协调。

Q: 如何保证 Feed 计数实时? A: 缓存只保存内容基础信息, 查询时从计数 SDS 刷新总数, 再按当前用户查 Bitmap 状态。计数事件还会根据反向索引旁路更新受影响页面, 但权威值仍以计数服务为准。

Q: 缓存一致性怎么讲才准确? A: 当前写操作前后都会主动删除详情缓存, 并通过反向索引清理本地 Feed 页; 这是尽量缩短不一致窗口, 不是强一致。更稳妥的方案是在事务提交后发布缓存失效事件, 所有实例订阅并失效 L1, 同时给 Redis 缓存保留 TTL 兜底。

6.5 搜索与 RAG

Q: 为什么用 search_after? A: ES 的 from + size 深分页需要每个分片保留并排序大量候选, 页数越深成本越高。search_after 带上上一页最后一条的排序值, 适合顺序翻页。排序末尾必须带唯一 ID, 避免相同分数和时间造成不稳定。

Q: RAG 如何避免跨文章污染? A: 当前是宽召回后在服务端按 postId 过滤, 只适合小数据量。正确改造是在向量查询阶段加入 metadata.postId 过滤, 让候选集从一开始就限定在当前文章, 并在结果中返回 chunkId/position 作为引用。

Q: 如何证明 RAG 变好了? A: 建立一组文章、问题、标准引用片段和可接受答案的数据集, 离线统计 Recall@K、MRR、引用命中率、Faithfulness 和 Answer Relevance, 在线记录首 token 延迟、总延迟、token 成本与空召回率。没有评测集就只能说“能跑”, 不能说“准确”。

6.6 安全与取舍

Q: JWT 登出后 Access Token 为什么还能用? A: Access Token 是 15 分钟左右的无状态令牌, 登出只撤销 Refresh Token, 旧 Access Token 会活到过期。这是性能与即时撤销之间的取舍; 高风险操作可增加 tokenVersion 或短期黑名单。

Q: 项目最大的不足是什么? A: 我会选“可靠性语义缺少闭环”而不是罗列小问题: 若异常被吞掉后仍确认上游位点, 就会把至少一次链路降级成可能丢消息; 同时缺少集成测试和可观测指标, 难以证明恢复能力。我的改造会围绕事件 ID、原子落地、重试/DLT、对账和 Testcontainers 展开。

7. 现有能力的简历写法

7.1 项目简介

知光是面向知识获取与分享场景的社区后端, 覆盖账号认证、内容渐进发布、Feed、点赞收藏、关注关系、全文搜索及单篇 RAG 问答。项目以 MySQL 保存核心业务事实, Redis 承接缓存与高频互动, 借助 Canal/Kafka 构建异步投影, 并通过 Elasticsearch 和 DeepSeek 提供搜索与流式知识问答。

7.2 负责功能 / 技术难点 (均可在当前源码中找到实现)

- 设计点赞/收藏计数链路: 用分片 Bitmap 保存用户行为事实, Lua 原子完成状态切换与幂等判定, Kafka 聚合增量后按秒折叠到固定结构 SDS, 兼顾高频写入与低延迟读。
- 为计数异常构建自愈路径: SDS 缺失或损坏时基于 Bitmap 分片重算, 并用 Redisson 锁、限流和指数退避抑制热点重建风暴; 批量查询通过 Redis Pipeline 降低网络往返。
- 将关注关系拆为主表和读投影: 在关注事务中写入 Outbox, 经 Canal/Kafka 异步维护粉丝表、ZSet 列表缓存和用户计数, 支持关系三态、时间游标分页与事实表重建。
- 拆分 Feed 页面顺序与内容片段缓存, 组合 Caffeine、Redis 和 MySQL 三级读取, 并加入空值缓存、TTL 抖动、热点续期、SingleFlight 与内容到页面的反向索引。
- 建立 Elasticsearch 内容检索: 使用 IK 分词、标题加权、标签过滤、业务字段 function score、高亮和 Completion Suggester, 并以 search_after 实现稳定深分页。
- 完成单篇知文 RAG 链路: 依据 SHA-256/ETag 判断版本, 按 Markdown 标题和重叠窗口切片写入向量库, 召回上下文后约束 DeepSeek 作答, 并以 SSE 流式返回结果。

使用提醒: 以上是“代码中已有机制”的技术概括, 不代表已经经过真实大流量生产验证。简历中最好补充你亲自完成的模块、测试结果或压测数据, 不要虚构指标。

8. 魔改方向: 按含金量排序

8.1 P0: 内容状态机 + 幂等发布 + 可靠 Outbox

为什么值得做

它同时解决业务正确性、事务一致性和异步可靠性, 面试官很容易追问, 工作量也可控。

建议设计

- 状态改为 DRAFT → CONTENT_READY → REVIEWING → PUBLISHED → DELETED/REJECTED;
- 所有状态转换收口到领域服务，SQL 使用 WHERE id=? AND status=? AND version=?;
- publish 接受幂等 Key，数据库用唯一索引保证同一请求只生效一次;
- 只有首次进入 PUBLISHED 才增加发文数并写 ContentPublished 事件;
- Outbox 插入失败必须让业务事务回滚，禁止吞异常;
- Outbox 增加 event_id/aggregate_version/created_at，消费者使用 inbox 唯一表幂等;
- Canal 到 Kafka 的桥接等待发送 Future 成功后再 ack，失败 rollback;
- 增加重试退避、死信 Topic、Outbox/Inbox 对账任务和消费延迟指标。

完成后可写进简历

建议简历功能点（完成对应改造后可写）：将内容发布收敛为带乐观锁的状态机，以幂等 Key 防止重复发布和重复计数；业务表与 Outbox 同事务提交，消费端借助 Inbox 唯一索引、重试与死信队列保证搜索/RAG 投影最终一致。

8.2 P0：计数链路可靠化 + 无阻塞重建

建议设计

- 将点赞 Lua 扩展为同一次原子操作内执行 SETBIT 与 XADD interaction-events;
- Redis Stream 消费成功后再投递 Kafka，或直接以 Consumer Group 聚合，消除 Bitmap 已变更但事件未落地的窗口;
- 事件加入唯一 event_id，聚合端记录去重状态;
- 聚合 Lua 在 HINCRBY 时同步 SADD counter:dirty，定时任务按批次 SPOP 脏实体，不再使用 KEYS agg:*;
- 创建 Bitmap 分片时维护 SADD bm:shards:{metric}:{entity}，重建按分片集合遍历;
- 增加负计数、事件积压、SDS 与 Bitmap 偏差、重建次数等指标;
- 用 Testcontainers 覆盖重复消息、乱序、消费崩溃、Redis 重启和重建并发。

完成后可写进简历

建议简历功能点（完成对应改造后可写）：在 Redis Lua 中原子写入 Bitmap 行为事实与 Stream 事件，消除状态成功但 Kafka 增量丢失的双写窗口；通过脏集合批量调度、事件去重和 Bitmap 对账实现计数链路的失败恢复。

8.3 P0：可评测、可引用的 RAG 2.0

建议设计

- 向量查询直接使用 metadata.postId 过滤，不在全局 Top N 后二次筛选；
- 使用 BM25 + 向量混合召回，按需要加入轻量 reranker；
- 切片元数据补充标题路径、段落序号、字符区间和内容版本；
- SSE 从纯文本升级为结构化事件：retrieval、token、citation、done/error；
- 索引改为异步任务状态机，先写新版本，成功后切换 active version，再删除旧版本，避免先删后写失败造成空窗；
- 建立 50~100 条人工评测集，输出 Recall@K、MRR、引用命中率、Faithfulness、Answer Relevance、P95 延迟和单问成本；
- 对匿名接口增加 IP/用户限流、问题长度限制、超时与取消传播。

完成后可写进简历

建议简历功能点（完成对应改造后可写）：构建文章级混合检索与引用溯源，使用元数据过滤、版本化索引和结构化 SSE 避免跨文章污染与重建空窗，并以固定评测集持续跟踪 Recall@K、忠实度、延迟和调用成本。

8.4 P1：关注流个性化 Feed（区分度最高，工作量较大）

建议设计

- 普通作者发布后，通过 Kafka 将内容 ID 推入粉丝时间线 ZSet；
- 大 V 不做全量 fan-out，查询时从其作者流拉取，采用推拉结合避免写放大；
- 读取时合并用户时间线、大 V 候选和公共热门池，使用 (score, contentId) 双字段游标去重分页；
- 排序分数可组合新鲜度、关注权重、互动热度和标签偏好，第一版只做可解释的规则；
- 发布、删除、改可见性都发送领域事件，Feed 消费端幂等更新；
- 增加时间线回填、过期清理和大 V 阈值动态调整。

完成后可写进简历

建议简历功能点（完成对应改造后可写）：设计关注流推拉结合架构，对普通作者异步 fan-out、对大 V 查询时 fan-in，通过 ZSet 时间线、稳定游标和幂等内容事件控制写放大，并支持删除与可见性变更的增量撤回。

8.5 P1：搜索索引工程化

建议设计

- 订阅点赞/收藏聚合事件，批量更新 ES 排序特征，或查询时从 Redis 取实时特征做二阶段排序；

- 建立 index_job 状态、重试次数、最后错误和 next_retry_at;
- 用版本化索引 + alias 完成全量重建和零停机切换;
- 搜索异常不再静默返回空结果, 区分 “确实无结果” 和 “搜索服务降级” ;
- 对游标加入版本和签名, 校验排序字段数量与类型;
- 压测关键词、标签过滤和深分页, 记录 P50/P95、慢查询与召回量。

8.6 P1: 会话安全与验证码原子化

建议设计

- 将 Refresh Token 校验与消费改成 Lua 原子操作;
- 引入 token family 与 reuse detection: 旧令牌再次出现时撤销整个设备会话族;
- 保存设备名、IP 摘要、最近活跃时间, 提供会话列表和强制下线;
- 验证码 “读取、比较、递增尝试次数、成功删除” 改为单个 Lua 脚本;
- 发送限流从 GET + SET 改为 SET NX EX, 日限额按自然日剩余秒数过期;
- RSA 私钥移出源码资源, 改用环境密钥、KMS 或 Secret 管理。

8.7 P1: 可观测性与验证体系

这项不够 “炫”, 但最能把前述设计变成可信成果。

- Micrometer 指标: 缓存命中率、回源次数、SingleFlight 等待数、Kafka lag、DLT 数、计数偏差、RAG 空召回率与模型耗时;
- 日志统一 traceId/eventId/postId/userId, 异步链路透传事件 ID;
- Testcontainers 拉起 MySQL、Redis、Kafka、ES, 验证主链路而非只 mock;
- k6/Gatling 压测点赞热点、Feed 热页、关注大 V 和 RAG 并发;
- 故障演练: Kafka 暂停、Redis 丢失 SDS、消费者处理到一半退出、ES 写入失败;
- 报告只写实测数据, 并说明硬件、数据量、并发模型和瓶颈。

9. 可参考的开源实现

以下项目只用于学习改造思路, 不是当前仓库已有能力, 也不能直接写进简历。

学习来源	值得看的机制	对知光的合理借鉴	不建议照搬
Debezium Outbox Example	业务库内写 Outbox、Debezium 捕获变更、Kafka 投递、消费端按事件 ID 排除重复消息，并接入 OpenTelemetry/Jaeger	给 Outbox 增加稳定 event_id 和聚合版本；消费投影与 Inbox 去重记录同事务提交；补充事件延迟与失败追踪	不必为了复刻示例把当前单体立即拆成多个服务，也不必更换成 Quarkus/PostgreSQL
Spring AI Evaluation Advisor Demo	将主模型生成与独立评估器分开，设置质量阈值与最大重试次数，失败时携带反馈重试	先建立固定问答/引用评测集，再把评测器作为补充；所有自动重试必须设置上限并记录成本、延迟和失败原因	不要只靠 LLM-as-a-Judge 宣称“准确率提升”，也不要没有离线基线时做无限自反思循环

对当前项目而言，最值得借鉴的是“可证明的可靠性与质量闭环”，不是把参考仓库的技术栈和目录结构整体搬过来。

10. 推荐落地路线

10.1 两周“稳妥增值版”

阶段	工作	可验证产物
第 1~3 天	发布状态机、乐观锁、幂等 Key、Outbox 异常回滚	状态转换表、数据库迁移、并发发布测试
第 4~6 天	Canal/Kafka 确认语义、event_id、Inbox 幂等、DLT	重复消息与发送失败集成测试
第 7~9 天	RAG 的 postId 服务端过滤、版本化索引、引用元数据	跨文章污染测试、引用返回示例
第 10~12 天	Testcontainers 与核心指标	一键集成测试、Prometheus 指标截图
第 13~14 天	2 个场景压测和复盘文档	P50/P95、错误率、瓶颈与优化前后对比

10.2 四周“面试区分度版”

在两周版基础上增加个性化 Feed 推拉结合、RAG 混合召回与评测集。不要同时拆微服务、上分库分表、做推荐模型和支付；范围过大会导致每个亮点都无法讲透。

10.3 建议优先级

- 面向 Java 后端岗位：状态机/可靠事件 → 计数可靠化 → Testcontainers/压测 → 个性化 Feed；
- 面向 AI 应用后端：RAG 评测与引用 → 异步索引状态机 → 可靠事件 → 成本与限流；
- 时间只有一周：先做状态机、修复公共缓存用户态污染、RAG 元数据过滤，再补对应测试。

11. 不建议为了简历硬加的内容

- 不要把当前单体机械拆成十几个微服务；没有独立扩缩容、团队边界和故障隔离需求时，只会增加部署负担；
- 不要为了“分布式”直接上分库分表，当前更缺的是可靠事件、测试与可观测性；
- 不要把容器化、配置中心、反向代理单独写成核心亮点，除非它们解决了明确的发布或故障问题；
- 不要虚构 QPS、数据量、命中率和准确率。先准备可重复的压测脚本与评测集，再写数字；
- 不要一次性把所有建议都塞进简历。选 3~4 条能画出数据流、说清失败场景、展示测试结果的改造即可。

12. 面试前自检清单

- 能否在两分钟内画出“发布、点赞、关注、RAG”四条数据流？
- 能否指出每条链路的事实源、派生数据、事务边界和最终一致性点？
- 能否解释 Bitmap 分片、SDS 布局、Kafka 手动确认、Outbox、ZSet 游标和 search_after？
- 能否坦诚说明当前消息可靠性、缓存多实例一致性和 RAG 过滤的边界？
- 是否至少完成一项魔改，并有数据库迁移、测试、指标或压测结果作为证据？
- 简历上的每个数字是否都有脚本、环境和原始结果可复现？

做到这些，项目就不只是“用了很多中间件”，而会呈现为一个能讨论业务建模、数据一致性、并发性能、失败恢复和 AI 工程质量的完整作品。